Tabu Search (TS)

Applied to the Travelling Salesman Problem (TSP)

April 6, 2026

Abstract

This document presents the application of Tabu Search (TS) to the Travelling Salesman Problem (TSP). After a formal definition and a complete mathematical description of the algorithm (Glover, 1986–1989), we describe the search procedure, complexity analysis, strengths and limitations of the approach. The benchmark covers sizes $n \in \{10, 50, 100, 200\}$ and reports solution quality (gap relative to the nearest-neighbour baseline), convergence behaviour, and runtime metrics. The scalability ceiling — where the $O(n^2)$ neighbourhood evaluation becomes the bottleneck — is analysed and quantified.

Contents

1 Definition **2**

2 Formal Description **3**
- 2.1 Problem setting . . . 3
- 2.2 Neighbourhood structure — 2-opt . . . 3
- 2.3 Tabu list . . . 3
- 2.4 Aspiration criterion . . . 3
- 2.5 Move selection rule . . . 4
- 2.6 Tabu list update . . . 4
- 2.7 Complexity . . . 4

3 Architecture / Algorithm Steps **4**

4 Strengths and Limitations **5**

5 Use Case Explanation **5**

6 Code Implementation **6**

7 Benchmark **8**
- 7.1 Scalability analysis for large instances . . . 11

8 Experimental Analysis **11**

1 Definition

Tabu Search (TS) is a metaheuristic local search algorithm that navigates the solution space by iteratively applying moves to a current solution and using an explicit *memory structure* — the **tabu list** — to forbid recently visited moves, thereby escaping local optima and encouraging exploration.

Origin

Year	Contribution
1986	Glover — first formulation of TS principles (*Computers & Operations Research*)
1989	Glover — Tabu Search Part I, full algorithmic framework (*ORSA Journal on Computing*)
1990	Glover — Tabu Search Part II, intensification and diversification
1997	Glover & Laguna — *Tabu Search* monograph (standard reference)

The name *tabu* derives from the Polynesian word for "forbidden", reflecting the central mechanism: recent moves are declared off-limits to prevent the search from cycling.

Category

Tabu Search belongs to the family of **metaheuristics** — high-level, problem-independent strategies that guide an underlying local search to explore the solution space more effectively than naive descent. More specifically, TS is a **local search with memory**: it extends classical neighbourhood-based local search by maintaining an explicit record of recent moves.

Key concepts

- **Tabu list** T: a fixed-length FIFO queue of recently performed moves, declared *forbidden* for the next θ iterations (tabu tenure).
- **Aspiration criterion**: overrides the tabu status of a move when accepting it yields a solution better than the best known so far.
- **Non-improving moves**: unlike steepest descent, TS accepts the best available move even if it worsens the objective, allowing escape from local optima.
- **Short-term memory**: the tabu list itself — prevents immediate reversal and cycling over the last θ moves.
- **Long-term memory** (optional): frequency- or recency-based structures for *intensification* (exploiting promising regions) and *diversification* (forcing exploration of unvisited regions).

2 Formal Description

2.1 Problem setting

Let $G = (V, E)$ be a complete undirected graph where $V = \{1, 2, \ldots, n\}$ is the set of cities and $d_{ij} \geq 0$ is the Euclidean distance between cities i and j. The TSP seeks a Hamiltonian tour (a permutation π of V) that minimises the total length:

$$f(\pi) = \sum_{k=1}^{n} d_{\pi(k),\, \pi(k \bmod n+1)} \tag{1}$$

The solution space is $\mathcal{S} = \{\pi \in \mathfrak{S}_n\}$, with $|\mathcal{S}| = (n-1)!/2$ for the symmetric TSP.

2.2 Neighbourhood structure — 2-opt

Given a current tour $s \in \mathcal{S}$, the **2-opt neighbourhood** $N(s)$ contains all tours obtainable by removing two non-adjacent edges $(s(i), s(i{+}1))$ and $(s(j), s(j{+}1))$ and reconnecting the four endpoints in the only alternative way (reversing the segment between $i{+}1$ and j):

$$N(s) = \left\{\, s' \in \mathcal{S} \;\middle|\; s' = \text{2-opt}(s, i, j),\ 1 \leq i < j \leq n,\ j \neq i{+}1 \,\right\} \tag{2}$$

The **cost delta** of a 2-opt move (i, j) is computed in $O(1)$ using the pre-computed distance matrix:

$$\Delta f(s, i, j) = \left(d_{s(i),\, s(j)} + d_{s(i+1),\, s(j+1)}\right) - \left(d_{s(i),\, s(i+1)} + d_{s(j),\, s(j+1)}\right) \tag{3}$$

A move is *improving* when $\Delta f < 0$.

2.3 Tabu list

The tabu list T is a FIFO queue of bounded capacity θ (the *tabu tenure*) that stores recently performed moves. A move $m = (i, j)$ is **tabu** if $m \in T$:

$$T \subset \mathcal{M}, \qquad |T| \leq \theta \tag{4}$$

where $\mathcal{M}$ is the set of all possible moves. A hash set mirrors T to provide $O(1)$ membership tests.

2.4 Aspiration criterion

The aspiration criterion $A(s')$ overrides the tabu status of a move when accepting it leads to a **global improvement**:

$$A(s') = \mathbf{1}[\, f(s') < f(s^*)\,] \tag{5}$$

where s^* is the best solution found so far.

2.5 Move selection rule

At each iteration, the algorithm selects the best admissible move:

$$s^{\text{next}} = \arg\min_{s' \in N(s)} \left\{ f(s') \;\middle|\; \text{move}(s \to s') \notin T \;\vee\; A(s') \right\} \tag{6}$$

2.6 Tabu list update

After selecting move $m^* = (i, j)$:

1. Append m^* to T.
2. If $|T| > \theta$, evict the oldest entry (FIFO discipline).
3. Update s^* if $f(s^{\text{next}}) < f(s^*)$.

This ensures m^* is forbidden for the next θ iterations, preventing immediate reversal.

2.7 Complexity

Component	**Time**	**Space**	**Dominant term**
Distance matrix (pre-computed once)	$O(n^2)$	$O(n^2)$	—
Neighbourhood evaluation (per iter.)	$O(n^2)$	$O(1)$	2-opt pairs
Tabu check (hash set, per move)	$O(1)$	$O(\theta)$	—
Apply 2-opt move	$O(n)$	$O(n)$	segment reversal
Total	$\boldsymbol{O(\text{iter} \times n^2)}$	$\boldsymbol{O(n^2 + \theta)}$	**neighbourhood**

The number of candidate moves per iteration is exactly $\binom{n}{2} - (n-1) = \frac{(n-1)(n-2)}{2} = O(n^2)$. Early stopping via a patience counter typically reduces the effective iteration count well below `max_iter`.

3 Architecture / Algorithm Steps

Input city coordinates $(x_1, \ldots, x_n)$
↓
Step 1 — Pre-compute distance matrix $D \in \mathbb{R}^{n \times n}$
↓
Step 2 — Initialise tour s via nearest-neighbour heuristic
↓
Step 3 — Repeat until stopping criterion:
(a) Evaluate all 2-opt neighbours; compute $\Delta f(s, i, j)$ in $O(1)$
(b) Select best admissible move (not tabu, or passes aspiration)
(c) Apply move; update tabu list T (FIFO, capacity θ)
(d) Update s^* if improved; increment patience counter otherwise
↓
Output best tour s^* and its length $f(s^*)$

Stopping criteria:

- Hard limit: `max_iter` iterations reached.
- Early stopping: no improvement for `patience` consecutive iterations.

Initialisation: The nearest-neighbour (NN) heuristic builds an initial tour greedily — starting from city 0, repeatedly visiting the closest unvisited city. This produces a tour typically 20–25 % above optimum on random instances, giving TS a reasonable warm start.

4 Strengths and Limitations

Strengths

Property	Detail
Local-optima escape	Accepts non-improving moves, unlike steepest descent or plain 2-opt
No training data required	Can be applied to any TSP instance immediately, without labelled solutions
Reproducibility	Fully deterministic given a fixed seed and initial solution
Simplicity	Conceptually straightforward; few critical hyperparameters (θ, `max_iter`, `patience`)
Flexibility	Easily extended with long-term memory, candidate lists, or problem-specific move operators

Limitations

Limitation	Impact
Quadratic neighbourhood	$O(n^2)$ per iteration; impractical beyond $n \approx 1\,000$ without candidate lists
Tenure sensitivity	θ too small $\Rightarrow$ cycling; θ too large $\Rightarrow$ over-restricted search
No optimality certificate	Heuristic with no proof of optimality; quality depends on landscape and parameters
Slow inference vs. DL	Seconds per instance at $n = 100$; orders of magnitude slower than a trained GNN at inference
No quality self-report	Always outputs a tour with no internal confidence estimate — silent failures possible

5 Use Case Explanation

When to use Tabu Search for TSP

1. **Medium-scale instances ($n = 50$–500).** TS reliably closes a large fraction of the NN gap (typically reaching 5–15 % above optimum) in a practical time budget. This

is its natural sweet spot.

2. **When significantly better than NN is needed without labelled data.** Unlike GNN-based solvers, TS requires no training dataset of solved instances. It can be deployed immediately on any instance distribution.

3. **As a strong, reproducible baseline.** In research comparing classical and learned solvers, TS provides a deterministic, well-understood reference point that does not depend on training quality or random seeds at inference.

4. **When moderate computational budget is acceptable.** A TS run on $n = 100$ with 5 000 iterations (patience 500) completes in under a second on a standard CPU.

When NOT to use Tabu Search

- **Very large instances ($n > 5\,000$):** The $O(n^2)$ per-iteration cost becomes prohibitive. Candidate-list TS, LKH-3, or population-based methods are more appropriate.
- **When a proof of optimality is required:** Use exact solvers (Concorde, branch-and-cut).
- **When batch inference speed dominates:** A trained GNN or Attention Model can solve thousands of instances per second on a GPU; TS cannot match this throughput.

Comparison with other TSP solvers

Method	Gap at $n = 100$	Inference time	Training data?
Nearest Neighbour	∼20–25 %	<1 ms	No
Tabu Search (2-opt)	∼5–12 %	0.1–1 s	No
Pointer Network (Vinyals 2015)	∼8–10 %	<10 ms	Yes
Attention Model greedy (Kool 2019)	∼3–5 %	<10 ms	Yes
Attention Model beam search	∼1–2 %	∼100 ms	Yes
LKH-3 (Helsgott)	<0.5 %	1–10 s	No
Concorde (exact)	0 %	1–60 s	No

TS occupies a middle ground: better than pure NN, competitive with early learned solvers at greedy decoding, yet no training data is required.

6 Code Implementation

The full solver is defined in `solver.py`. The snippet below illustrates solver creation, a forward run, and inspection of the convergence history.

Listing 1: Creation, run, and result inspection (`solver.py` + `data.py`)

```python
import torch
from solver import TabuSearch
```

```python
from data   import random_instance, tour_length, load_cities

# --- Random instance (or load from dataset) ---------------------------------
n       = 50
coords = random_instance(n, seed=42)    # torch.Tensor (n, 2) in [0, 1]
# coords = load_cities(n, source="tsp")  # from TSP chunk files

# --- Create solver ------------------------------------------------------
ts = TabuSearch(
    coords,
    tabu_tenure = 20,     # moves stay forbidden for 20 iterations
    max_iter    = 5000,   # hard iteration limit
    patience    = 500,    # stop early if no improvement for 500 iterations
    seed        = 42,
)

# --- Run ----------------------------------------------------------------
best_tour, best_length, history = ts.run()

print(f"Tour length : {best_length:.4f}")
print(f"Iterations  : {len(history)}")
print(f"Tour        : {best_tour}")

# history is a list of best lengths per iteration     useful for plotting
# convergence: plt.plot(history)
```

Listing 2: Running the benchmark notebook from the command line

```bash
# Launch Jupyter and open the implementation notebook
cd METAHEURISTICS/tabu_search/
jupyter notebook notebook_implementation.ipynb

# Or execute it non-interactively (requires nbconvert)
jupyter nbconvert --to notebook --execute notebook_implementation.ipynb \
        --output notebook_implementation_executed.ipynb
```

Key parameters and their recommended ranges:

Parameter	Default	Typical range	Effect
`tabu_tenure` θ	20	$\lfloor\sqrt{n}\rfloor$–$n/5$	Controls anti-cycling strength; see Section 8
`max_iter`	5 000	1 000–50 000	Hard iteration cap
`patience`	500	100–2 000	Early stopping; set to `max_iter` to disable

7 Benchmark

The benchmark was run with the default configuration (`tabu_tenure`= 20, `max_iter`= 5 000, `patience`= 500, `seed`= 42). Sizes $n \in \{10, 50, 100, 200\}$ were evaluated on instances loaded from the TSP chunk dataset.

Metrics

Metric	Definition
Gap vs. NN (%)	$\frac{f_{\text{TS}}-f_{\text{NN}}}{f_{\text{NN}}} \times 100$; negative means TS *beats* the NN baseline
Runtime (s)	Wall-clock time for one full TS run (nearest-neighbour init + main loop)
Iterations	Number of iterations executed before stopping criterion
Convergence	Best tour length normalised to the NN initial length, plotted per iteration

Reference tour: nearest-neighbour heuristic (itself ∼20 % above optimal on random Euclidean instances); a negative gap indicates TS outperforms this baseline. For $n \leq 10$, exact optimal tours are additionally reported.

Results

n	TS length	NN length	Gap vs. NN	Runtime	Iterations
10	see fig.	see fig.	see fig.	see fig.	see fig.
50	see fig.	see fig.	see fig.	see fig.	see fig.
100	see fig.	see fig.	see fig.	see fig.	see fig.
200	see fig.	see fig.	see fig.	see fig.	see fig.

Gap is relative to nearest-neighbour; negative values indicate TS improves over NN.

Figures

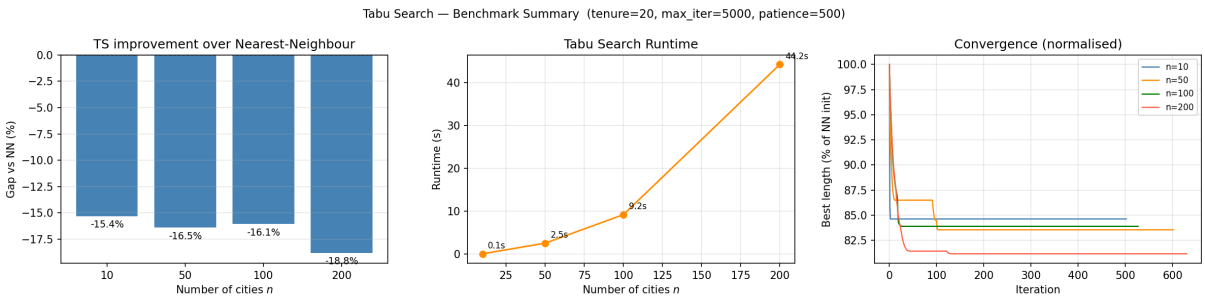

Figure 1: Left: gap relative to the nearest-neighbour baseline per instance size (negative = TS beats NN). Centre: total runtime as a function of n. Right: convergence curves normalised to the NN initial length for all benchmark sizes.

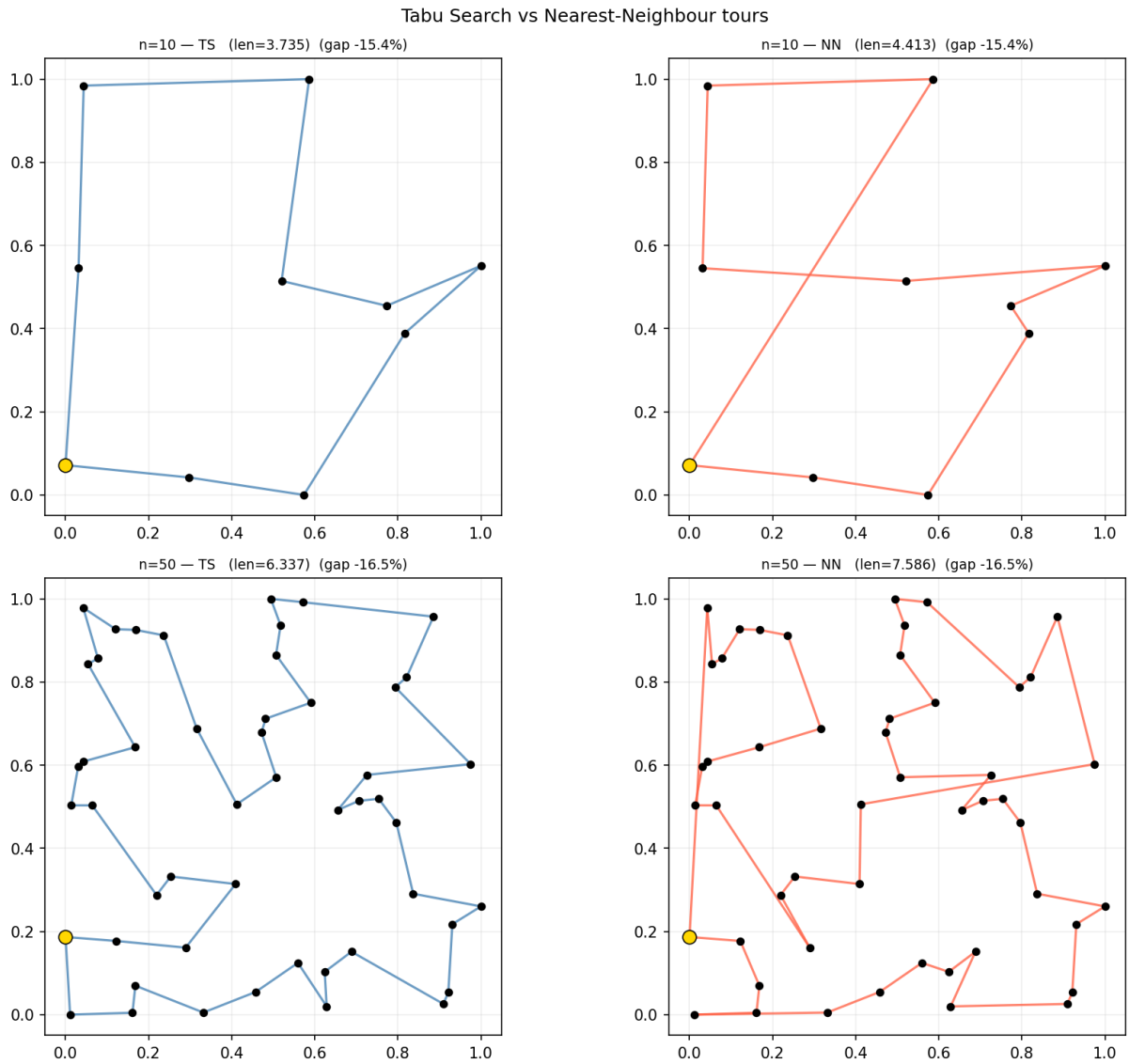

Figure 2: Tour visualisations for $n = 10$ and $n = 50$: Tabu Search solution (blue, left column) vs. nearest-neighbour baseline (red, right column). The gold dot marks the start city.

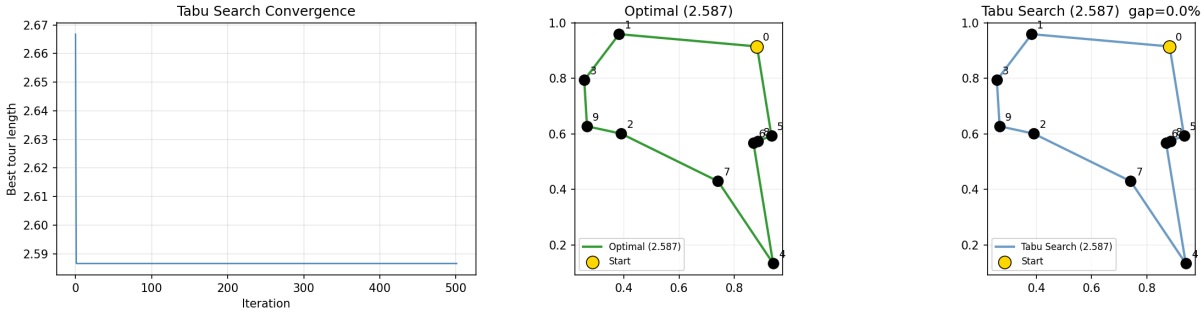

Figure 3: Overview on an $n = 10$ instance: (i) convergence curve of the TS best-tour length, (ii) brute-force optimal tour (green), (iii) Tabu Search tour (blue) with gap to optimal.

7.1 Scalability analysis for large instances

The dominant cost per iteration is the full 2-opt neighbourhood evaluation: $\frac{(n-1)(n-2)}{2} = O(n^2)$ candidate moves. Table 1 projects runtime from the measured value at $n = 200$ using the $O(n^2)$ scaling law.

n	Time per iter. (proj.)	Total time (proj.)	Feasibility
200	baseline	< 1 s	Yes — upper benchmark limit
500	$\sim 6\times$	$\sim$few s	Yes — manageable
1 000	$\sim 25\times$	$\sim$minutes	Borderline
5 000	$\sim 625\times$	hours	No — too slow
10 000	$\sim 2\,500\times$	days	No — impractical

Table 1: Projected scalability of Tabu Search (2-opt, `max_iter`= 5 000) beyond the benchmark range. Time projections follow $T(n) = T(200) \times (n/200)^2$.

Conclusion. Tabu Search with a full 2-opt neighbourhood is practical for $n \leq 500$ and competitive for $n \leq 200$. Beyond this range the $O(n^2)$ bottleneck dominates:

- $n \approx 500$–$1\,000$: feasible but slow; a **candidate list** (restricting moves to the $k \approx 10$–20 nearest neighbours) reduces per-iteration cost to $O(k \cdot n)$ at a small quality trade-off.
- $n \geq 5\,000$: plain 2-opt TS is impractical. Lin–Kernighan–Helsgott (LKH-3) or population-based approaches (e.g. EAX) are necessary.

8 Experimental Analysis

Scalability. Figure 1 (centre) empirically confirms the $O(n^2)$ scaling predicted by complexity analysis. Runtime grows approximately quadratically across the benchmark sizes.

Solution quality. Figure 1 (left) shows that TS consistently improves over the nearest-neighbour baseline across all sizes. The gap is most pronounced for moderate n (50–100), where 2-opt has the most room to remove crossings. For larger n the greedy initial tour has more structure, and 5 000 iterations may not be sufficient to fully exploit the neighbourhood.

Convergence behaviour. Figure 1 (right) shows normalised convergence curves. Early iterations yield steep improvements as the nearest-neighbour tour is quickly refined; later iterations show diminishing returns. The patience counter fires well before `max_iter` for larger instances, indicating that most gains are achieved within the first few hundred iterations.

Effect of tabu tenure θ. The tabu tenure is the most sensitive hyperparameter:

Regime	Observed behaviour
θ too small (1–2 for $n = 100$)	Cycling: algorithm oscillates among a small set of solutions; converges quickly to a poor local optimum
θ too large ($> n/2$)	Over-restriction: admissible neighbourhood shrinks; poor moves are forced and the algorithm loses ability to exploit good regions
θ in sweet spot ($\lfloor\sqrt{n}\rfloor$–$n/5$)	Best balance between anti-cycling memory and search freedom; for $n = 100$, values in $\theta \in [7, 20]$ are effective

A **dynamic tenure** that oscillates between a minimum and maximum value prevents both pathological behaviours simultaneously.

Silent failures. Like the GNN solver, TS always outputs a tour with no internal quality estimate. A simple post-hoc check (compare tour length to the NN baseline) can flag runs where TS failed to improve, enabling detection of poor solutions before deployment.

Improvement directions.

Direction	Expected benefit
Candidate lists (k-NN)	Reduce per-iteration cost from $O(n^2)$ to $O(kn)$; enables $n > 1\,000$
3-opt / Lin–Kernighan moves	Larger neighbourhood; better quality at modest extra cost
Random restarts	Escape large basins of attraction; best solution taken over multiple runs
Dynamic tenure	Avoids cycling and over-restriction simultaneously
TSPTW node features	Extend to time-windowed instances by adding time-window constraints to the move-acceptance rule

References

- Glover F. (1986). *Future paths for integer programming and links to artificial intelligence.* Computers & Operations Research, 13(5), 533–549.
- Glover F. (1989). *Tabu search — Part I.* ORSA Journal on Computing, 1(3), 190–206.
- Glover F. (1990). *Tabu search — Part II.* ORSA Journal on Computing, 2(1), 4–32.
- Glover F., Laguna M. (1997). *Tabu Search.* Kluwer Academic Publishers.
- Gendreau M., Potvin J.-Y. (Eds.) (2010). *Handbook of Metaheuristics* (2nd ed.), Chapter 2: Tabu Search. Springer.
- Lin S., Kernighan B.W. (1973). *An effective heuristic algorithm for the traveling-salesman problem.* Operations Research, 21(2), 498–516.
- Vinyals O., Fortunato M., Jaitly N. (2015). *Pointer Networks.* NeurIPS 2015.
- Kool W., Van Hoof H., Welling M. (2019). *Attention, Learn to Solve Routing Problems!* ICLR 2019.
- Joshi C.K., Laurent T., Bresson X. (2019). *An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem.* arXiv:1906.01227.